Seminar 2 - Conectare DB Oracle & Operatiuni cu DB

Cuprins

1. Tabele in baza de date
2. Configurare Maven si adaugare OJDBC
3. Conectare la baza de date
4. Clase de mapare (model)
5. Operatii JDBC cu try-with-resources
6. Exercitii

> ⚠️ NOTA
>
> Fiecare student va realiza exemplele de mai jos pe calculatorul lui si de asemenea exercitiile de la final. Tot codul sursa pentru exercitiile individuale de la finalul documentului va fi incarcat pe online.ase.ro in sectiunea dedicata pana la finalul seminarului in vederea obtinerii unui punctaj.

1. Tabele in baza de date

- Vom lucra cu un domeniu bancar simplificat — conturi si tranzactii.
- Ruleaza scripturile de mai jos in SQL Developer sau orice client Oracle.

```sql
-- Tabela de conturi bancare
CREATE TABLE accounts (
    id          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    iban        VARCHAR2(34)   NOT NULL UNIQUE,
    owner_name  VARCHAR2(100)  NOT NULL,
    balance     NUMBER(15, 2)  DEFAULT 0 NOT NULL,
    currency    VARCHAR2(3)    DEFAULT 'RON' NOT NULL,
    status      VARCHAR2(10)   DEFAULT 'ACTIVE' NOT NULL,
    created_at  DATE           DEFAULT SYSDATE NOT NULL,
    CONSTRAINT chk_status CHECK (status IN ('ACTIVE', 'BLOCKED', 'CLOSED')),
    CONSTRAINT chk_balance CHECK (balance >= 0)
);

-- Tabela de tranzactii
CREATE TABLE transactions (
    id              NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    account_id      NUMBER         NOT NULL,
    type            VARCHAR2(6)    NOT NULL,
    amount          NUMBER(15, 2)  NOT NULL,
    description     VARCHAR2(255),
    transaction_date DATE          DEFAULT SYSDATE NOT NULL,
    CONSTRAINT fk_account FOREIGN KEY (account_id) REFERENCES accounts(id),
    CONSTRAINT chk_type   CHECK (type IN ('CREDIT', 'DEBIT')),
    CONSTRAINT chk_amount CHECK (amount > 0)
);

-- Date de test
INSERT INTO accounts (iban, owner_name, balance, currency) VALUES
    ('RO49AAAA1B31007593840000', 'Ion Popescu',   5000.00, 'RON');
INSERT INTO accounts (iban, owner_name, balance, currency) VALUES
    ('RO49BBBB1B31007593840001', 'Maria Ionescu', 12000.00, 'RON');
INSERT INTO accounts (iban, owner_name, balance, currency) VALUES
    ('RO49CCCC1B31007593840002', 'Andrei Rusu',   0.00, 'RON');

INSERT INTO transactions (account_id, type, amount, description) VALUES (1,
'CREDIT', 1000.00, 'Salariu');
INSERT INTO transactions (account_id, type, amount, description) VALUES (1,
'DEBIT',   200.00, 'Factura curent');
INSERT INTO transactions (account_id, type, amount, description) VALUES (2,
'CREDIT', 5000.00, 'Transfer primit');
INSERT INTO transactions (account_id, type, amount, description) VALUES (2,
'DEBIT',   350.00, 'Cumparaturi');

COMMIT;
```

2. Configurare Maven si adaugare OJDBC

- Maven este un build tool folosit pentru management de dependinte, structura de proiect, build lifecycle.
- La un proiect Java deja creat se adauga fisierul `pom.xml` la nivel de `root` (acelasi nivel cu folderul `src`).

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         http://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.bdsa</groupId>
    <artifactId>BDSA</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <dependency>
            <groupId>com.oracle.database.jdbc</groupId>
            <artifactId>ojdbc8</artifactId>
            <version>21.9.0.0</version>
        </dependency>
    </dependencies>
</project>
```

- In IntelliJ o sa apara un popup `Load Maven` si vom da click pe el.
- In Maven `src` folder este definit ca `src/main/java` si vom crea aceeasi ierarhie si vom muta clasa `Main` in `../java`
- In cadrul `<dependencies>` se pot adauga `<dependency>` cu JAR-urile externe de care avem nevoie, in cazul nostru `OJDBC` pentru conectarea la Oracle DB.

3. Conectare la baza de date

- JDBC foloseste `DriverManager` pentru a crea o conexiune pe baza unui **connection string** (URL) si a credentialelor.

3.1. Formatul URL pentru Oracle

```
jdbc:oracle:thin:@//host:port/serviceName
```

- `thin` — driverul pur Java (nu necesita Oracle Client instalat)
- `host` — adresa serverului Oracle (ex. `localhost`)
- `port` — portul implicit Oracle este `1521`
- `serviceName` — numele serviciului (ex. `XEPDB1` pentru Oracle XE, `ORCL` pentru instalari standard)

3.2. Clasa `DatabaseConfig`

- Pentru a centraliza credentialele bazei de date si pentru a prelua conexiunea cat mai usor, vom folosi o clasa dedicata.

```java
public class DatabaseConfig {
    private static final String URL      =
"jdbc:oracle:thin:@//localhost:1521/XEPDB1";
    private static final String USERNAME = "tutorial_user";
    private static final String PASSWORD = "parola_ta";

    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL, USERNAME, PASSWORD);
    }
}
```

> ⚠️ **Nu hardcoda credentialele in cod de productie.** Foloseste variabile de mediu sau un fisier `.properties` ignorat de `.gitignore`. In scop didactic, centralizarea intr-o singura clasa e suficienta.

3.3. Testare conexiune

```
public class ConnectionTest {
    public static void main(String[] args) {
        try (Connection conn = DatabaseConfig.getConnection()) {
            System.out.println("Conexiune reusita: " +
conn.getMetaData().getURL());
        } catch (SQLException e) {
            System.err.println("Eroare conexiune: " + e.getMessage());
        }
    }
}
```

- Daca rulezi si vezi `Conexiune reusita: jdbc:oracle:thin:...` — esti gata sa continui.

4. Clase de mapare (model)

- Clasele de mapare (numite si **POJO** — Plain Old Java Object sau **entity**) reprezinta in Java structura unui rand din baza de date. Un camp din tabela devine un camp in clasa, tipurile de date se mapeaza corespunzator.

Mapare tipuri Oracle → Java

Oracle	Java
`NUMBER`	`int`, `long`, `double`, `BigDecimal`
`VARCHAR2`	`String`
`DATE`	`java.time.LocalDate`
`TIMESTAMP`	`java.time.LocalDateTime`
`CLOB`	`String`

4.1. Clasa `Account`

```
import java.time.LocalDate;

public class Account {
    private Long id;
    private String iban;
    private String ownerName;
    private double balance;
    private String currency;
    private String status;
    private LocalDate createdAt;

    public Account() {}

    public Account(String iban, String ownerName, double balance, String currency)
{
        this.iban = iban;
        this.ownerName = ownerName;
        this.balance = balance;
        this.currency = currency;
    }

    // Getters & Setters
    // toString
}
```

4.2. Clasa `Transaction`

```
import java.time.LocalDate;

public class Transaction {
    private Long id;
    private Long accountId;
    private String type;
    private double amount;
    private String description;
    private LocalDate transactionDate;

    public Transaction() {}

    public Transaction(Long accountId, String type, double amount, String
description) {
        this.accountId = accountId;
        this.type = type;
        this.amount = amount;
        this.description = description;
    }

    // Getters & Setters
    // toString
}
```

5. Operatii JDBC cu try-with-resources

De ce try-with-resources?

`Connection`, `PreparedStatement` si `ResultSet` implementeaza `AutoCloseable` — daca le deschizi, **trebuie** sa le inchizi. Daca uiti, ramai cu conexiuni deschise pe server pana la timeout. `try-with-resources` garanteaza ca `close()` e apelat automat, chiar si la exceptie.

```
// Pattern de baza
try (Connection conn = DatabaseConfig.getConnection();
     PreparedStatement ps = conn.prepareStatement("SELECT ...")) {
    // lucreaza cu conn si ps
} // conn.close() si ps.close() sunt apelate automat aici
```

> ⚠️ **Intotdeauna foloseste `PreparedStatement` in loc de `Statement`.**
>
> `Statement` cu concatenare de String-uri (`"SELECT * FROM accounts WHERE id = " + id`) este vulnerabil la **SQL Injection**. `PreparedStatement` cu parametri (`?`) trimite query-ul si valorile separat — baza de date nu poate confunda datele cu instructiuni SQL.

5.1. SELECT — citire date

Selectare toate conturile:

```
public List<Account> findAll() throws SQLException {
    String sql = "SELECT id, iban, owner_name, balance, currency, status,
created_at FROM accounts";
    List<Account> accounts = new ArrayList<>();

    try (Connection conn = DatabaseConfig.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql);
         ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            Account acc = new Account();
            acc.setId(rs.getLong("id"));
            acc.setIban(rs.getString("iban"));
            acc.setOwnerName(rs.getString("owner_name"));
            acc.setBalance(rs.getDouble("balance"));
            acc.setCurrency(rs.getString("currency"));
            acc.setStatus(rs.getString("status"));
            acc.setCreatedAt(rs.getDate("created_at").toLocalDate());
            accounts.add(acc);
        }
    }

    return accounts;
}
```

Selectare cont dupa ID (cu parametru):

```
public Optional<Account> findById(long id) throws SQLException {
    String sql = "SELECT id, iban, owner_name, balance, currency, status,
created_at " +
                 "FROM accounts WHERE id = ?";

    try (Connection conn = DatabaseConfig.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setLong(1, id); // primul ? devine valoarea id

        try (ResultSet rs = ps.executeQuery()) {
            if (rs.next()) {
                Account acc = new Account();
                acc.setId(rs.getLong("id"));
                acc.setIban(rs.getString("iban"));
                acc.setOwnerName(rs.getString("owner_name"));
                acc.setBalance(rs.getDouble("balance"));
                acc.setCurrency(rs.getString("currency"));
                acc.setStatus(rs.getString("status"));
                acc.setCreatedAt(rs.getDate("created_at").toLocalDate());
                return Optional.of(acc);
            }
        }
    }

    return Optional.empty();
}
```

> `rs.next()` avanseaza cursorul la urmatorul rand si returneaza `false` cand nu mai sunt randuri. La inceput cursorul e pozitionat **inainte** de primul rand — de aceea primul apel `rs.next()` muta la randul 1.

5.2. INSERT — adaugare inregistrare

```
public static long insert(Account account) throws SQLException {
    String sql = "INSERT INTO accounts (iban, owner_name, balance, currency)
VALUES (?, ?, ?, ?)";
    String[] generatedColumns = {"ID"};

    // in generatedColumns putem prelua ID-urie inapoi prin mecanismul de mai jos
generatedKeys

    try (Connection conn = DatabaseConfig.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql, generatedColumns))
    {
        ps.setString(1, account.getIban());
        ps.setString(2, account.getOwnerName());
        ps.setDouble(3, account.getBalance());
        ps.setString(4, account.getCurrency());
        int rowsAffected = ps.executeUpdate();
        System.out.println("Randuri inserate: " + rowsAffected);

        // preluare ID generat automat
        try (ResultSet generatedKeys = ps.getGeneratedKeys()) {
            if (generatedKeys.next()) {
                return generatedKeys.getLong(1);
            }
        }
    }

    throw new SQLException("INSERT esuat, nu s-a generat niciun ID.");
}
```

5.3. UPDATE si DELETE — cu verificare rowCount

La UPDATE si DELETE este important sa verifici cate randuri au fost afectate — daca 0, inseamna ca inregistrarea nu exista sau conditia nu a potrivit nimic.

UPDATE — modificare sold:

```
public static boolean updateBalance(long accountId, double newBalance) throws
SQLException {
    String sql = "UPDATE accounts SET balance = ? WHERE id = ? AND status =
'ACTIVE'";

    try (Connection conn = DatabaseConfig.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql))
    {
        ps.setDouble(1, newBalance);
        ps.setLong(2, accountId);

        return ps.executeUpdate() > 0;
    }
}
```

DELETE — stergere tranzactie:

```java
public boolean deleteTransaction(long transactionId) throws SQLException {
    String sql = "DELETE FROM transactions WHERE id = ?";

    try (Connection conn = DatabaseConfig.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql))
    {
        ps.setLong(1, transactionId);

        return ps.executeUpdate() > 0;
    }
}
```

5.4. CallableStatement — proceduri si functii Oracle

`CallableStatement` este folosit pentru a apela proceduri stocate (`PROCEDURE`) si functii (`FUNCTION`) din Oracle PL/SQL.

Sintaxa de apel:

```
// Procedura: { call nume_procedura(?, ?) }
// Functie:   { ? = call nume_functie(?) }
```

Exemplu — procedura care face un transfer intre conturi:

Mai intai, procedura PL/SQL:

```
CREATE OR REPLACE PROCEDURE transfer_funds(
    p_from_id  IN  NUMBER,
    p_to_id    IN  NUMBER,
    p_amount   IN  NUMBER,
    p_result   OUT VARCHAR2
) AS
    v_balance NUMBER;
BEGIN
    SELECT balance INTO v_balance FROM accounts WHERE id = p_from_id FOR UPDATE;

    IF v_balance < p_amount THEN
        p_result := 'INSUFFICIENT_FUNDS';
        RETURN;
    END IF;

    UPDATE accounts SET balance = balance - p_amount WHERE id = p_from_id;
    UPDATE accounts SET balance = balance + p_amount WHERE id = p_to_id;

    INSERT INTO transactions (account_id, type, amount, description)
        VALUES (p_from_id, 'DEBIT', p_amount, 'Transfer out');
    INSERT INTO transactions (account_id, type, amount, description)
        VALUES (p_to_id, 'CREDIT', p_amount, 'Transfer in');

    COMMIT;
    p_result := 'SUCCESS';
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        p_result := 'ERROR: ' || SQLERRM;
END;
/
```

Apel din Java:

```java
public String transferFunds(long fromId, long toId, double amount) throws
SQLException {
    String sql = "{ call transfer_funds(?, ?, ?, ?) }";

    try (Connection conn = DatabaseConfig.getConnection();
         CallableStatement cs = conn.prepareCall(sql))
    {
        // parametri IN
        cs.setLong(1, fromId);
        cs.setLong(2, toId);
        cs.setDouble(3, amount);

        // parametru OUT — trebuie inregistrat cu tipul sau Oracle
        cs.registerOutParameter(4, Types.VARCHAR);

        cs.execute();

        // citire valoare OUT dupa execute()
        String result = cs.getString(4);
        return result;
    }
}
```

Exemplu — functie care returneaza soldul unui cont:

```oracle
CREATE OR REPLACE FUNCTION get_balance(p_account_id IN NUMBER)
RETURN NUMBER AS
    v_balance NUMBER;
BEGIN
    SELECT balance INTO v_balance FROM accounts WHERE id = p_account_id;
    RETURN v_balance;
EXCEPTION
    WHEN NO_DATA_FOUND THEN RETURN -1;
END;
/
```

Apel din Java:

```java
public double getBalance(long accountId) throws SQLException {
    String sql = "{ ? = call get_balance(?) }";

    try (Connection conn = DatabaseConfig.getConnection();
         CallableStatement cs = conn.prepareCall(sql)) {

        // primul ? este valoarea returnata de functie
        cs.registerOutParameter(1, Types.NUMERIC);
        cs.setLong(2, accountId);

        cs.execute();

        return cs.getDouble(1);
    }
}
```

5.5. Cursor Oracle — preluare si parcurgere

Un **cursor** in Oracle e un pointer catre un result set returnat dintr-o procedura. In Java se mapeaza cu `Types.REF_CURSOR`.

Procedura cu cursor:

```
CREATE OR REPLACE PROCEDURE get_transactions_by_account(
    p_account_id IN  NUMBER,
    p_cursor     OUT SYS_REFCURSOR
) AS
BEGIN
    OPEN p_cursor FOR
        SELECT id, account_id, type, amount, description, transaction_date
        FROM transactions
        WHERE account_id = p_account_id
        ORDER BY transaction_date DESC;
END;
/
```

Preluare si parcurgere cursor din Java:

```
public List<Transaction> getTransactionsByAccount(long accountId) throws
SQLException {
    String sql = "{ call get_transactions_by_account(?, ?) }";
    List<Transaction> result = new ArrayList<>();

    try (Connection conn = DatabaseConfig.getConnection();
         CallableStatement cs = conn.prepareCall(sql))
    {

        cs.setLong(1, accountId);
        cs.registerOutParameter(2, OracleTypes.CURSOR); // din
oracle.jdbc.OracleTypes

        cs.execute();

        // cursorul e un ResultSet — se inchide automat la iesirea din try
        try (ResultSet rs = (ResultSet) cs.getObject(2)) {
            while (rs.next()) {
                Transaction t = new Transaction();
                t.setId(rs.getLong("id"));
                t.setAccountId(rs.getLong("account_id"));
                t.setType(rs.getString("type"));
                t.setAmount(rs.getDouble("amount"));
                t.setDescription(rs.getString("description"));

t.setTransactionDate(rs.getDate("transaction_date").toLocalDate());
                result.add(t);
```

```
            }
        }
    }

    return result;
}
```

> Pentru `OracleTypes.CURSOR` ai nevoie de importul:
>
> ```
> import oracle.jdbc.OracleTypes;
> ```
>
> Acesta vine din dependinta `ojdbc8` adaugata in `pom.xml` — nu trebuie nimic extra.

6. Exercitii

A. SELECT si mapare

Scrie o clasa `AccountRepository` cu urmatoarele metode:

1. `Optional<Account> findByIban(String iban)` — cauta un cont dupa IBAN.
2. `List<Account> findByStatus(String status)` — returneaza toate conturile cu un anumit status (`ACTIVE`, `BLOCKED`, `CLOSED`).
3. In `main`, apeleaza toate metodele si afiseaza rezultatele.

> HINT: Pentru `findByIban`, foloseste `WHERE iban = ?` cu `ps.setString(1, iban)`. Daca `rs.next()` returneaza `false`, intoarce `Optional.empty()`.

B. UPDATE & DELETE cu rowCount

Extinde `AccountRepository` cu:

1. `boolean block(long accountId)` — seteaza `status = 'BLOCKED'` pentru contul dat. Returneaza `false` daca contul nu exista sau e deja blocat.
2. `boolean delete(long accountId)` — sterge contul doar daca `balance = 0`. Returneaza `true` doar daca contul exista si soldul a fost 0 si a putut fi sters, altfel `false`. Verifica `rowsAffected`.

> HINT: La DELETE, scrie conditia direct in SQL: `DELETE FROM accounts WHERE id = ? AND balance = 0`. Daca `rowsAffected == 0`, fie ID-ul nu exista, fie soldul nu e 0 — nu stii exact care, dar poti face un SELECT inainte daca vrei sa diferentiezi cazurile.

> HINT: Puteti demonstra ca nu se sterge un cont care are balance > 0. Daca vreti sa testati si stergerea unui cont pe bune (care are balance 0), trebuie sa tineti cont ca tabelele `accounts` si `transactions` sunt legate prin `Foreign Keys`. Prin urmare, pentru a sterge un cont prima data trebuie sa-i stergem tranzactiile.

C. CallableStatement si Cursor

1. Scrie procedura PL/SQL `get_active_accounts(p_cursor OUT SYS_REFCURSOR)` care returneaza toate conturile cu `status = 'ACTIVE'`.
2. Scrie functia PL/SQL `count_transactions(p_account_id IN NUMBER) RETURN NUMBER` care returneaza numarul de tranzactii pentru un cont.
3. In Java, scrie metodele corespunzatoare (extinzand `AccountRepository`):
   - `List<Account> getActiveAccounts()` — apeleaza procedura si parcurge cursorul.
   - `int countTransactions(long accountId)` — apeleaza functia si returneaza rezultatul.
4. In `main`, pentru fiecare cont activ, afiseaza numarul sau de tranzactii.

> HINT: Pentru a combina cele doua: ia lista din `AccountRepository.getActiveAccounts()`, itereaza cu for-each si pentru fiecare `account` apeleaza `AccountRepository.countTransactions(account.getId())`. Doua metode separate, combinate in `main` cu `Stream` sau `for-each clasic`.

Bonus — Tranzactie manuala

- Implementeaza metoda `void transfer(long fromId, long toId, double amount)` **fara procedura stocata**, direct din Java, folosind tranzactie manuala JDBC:

```
conn.setAutoCommit(false); // dezactiveaza auto-commit. Autocommit-ul se face dupa
fiecare apel de callable statement sau prepared statement

try (conn, ps etc...){
    // 1. Verifica sold sursa, daca e indisponibil arunca
InsufficientFundsException (extinde SQLException)
    // 2. UPDATE balance - amount pe contul sursa
    // 3. UPDATE balance + amount pe contul destinatie
    // 4. INSERT in transactions pentru ambele (una CREDIT si una DEBIT)

    conn.commit(); // daca totul este cu succes, commitam toate modificarile,
altfel rollack - A(CID) - atomicitatea tranzactiilor
} catch (SQLException e) {
    conn.rollback(); // anuleaza tot daca ceva esueaza
    throw e;
}
```

- Verifica `rowsAffected` la fiecare UPDATE. Daca soldul e insuficient, arunca o exceptie custom `InsufficientFundsException` inainte de orice UPDATE si fa `rollback`.

Asist. univ. **Sabin Tarbă**
Email: **sabin.tarba@csie.ase.ro**